ログミー連載まとめ (全5回)

登壇者: 成瀬 允宣 氏（なるセミ）

Javaでクリーンアーキテクチャする方法

～現場の泥臭い課題と実践的な解決策～

「API未完成、DB未選定、でも納期は決まっている」という新規開発の現実の中で、 いかにビジネスロジックを中心に据えて試行錯誤を可能にするかを解説した人気講演のエッセンスまとめです。

Part.1 ヘキサゴナルアーキテクチャ：現実の泥沼とゲーム機の例え

元記事 ↗

💡 開発現場のリアル

理想は「DB → API → フロント」の順で作ること。しかし現実は「APIもDBも決まってないのにフロントを作り始めろと言われ、デザインも試行錯誤が必要」な状況に直面する。 この試行錯誤を耐え抜くためにアーキテクチャが必要。

🎮 Nintendo Switchに例えると一発でわかる

- **Switch本体:** アプリケーション（ビジネスロジックの核）
- **Joy-Con / プロコン:** Driving Adapter（操作入力口。入れ替え可能）
- **ドック / TV画面:** Driven Adapter（映像出力先。ポータブル液晶でもTVでも動く）

本体はゲームを動かすことだけに専念し、入力装置や出力装置（WebやDB）はすべて外側の「プラグイン」として切り離す。

Part.2 クリーンアーキテクチャを理解する：同心円の「右下」を見よ

元記事 ↗

1. Entities (中心)

ER図のテーブル定義ではない。ビジネスルールそのものをカプセル化した純粋なドメインオブジェクト。

2. Use Cases

ドメインを使って、ユーザーの問題を解決するアプリケーションの具体的なフロー。

3. Interface Adapters

Controller, Presenter, Gateway。外の世界のデータと内側のデータを相互変換する。

4. Frameworks & Drivers
Spring、PostgreSQL、Webブラウザなどの最も変わりやすい詳細技術。

🎯 **最重要ポイント:** 有名な「同心円の図」だけでなく、**右下に小さく描かれている「フロー図（Controller → UseCase → Presenter）」**こそが実装の設計図である。

Part.3 実装例：SOLID原則を突き詰めた部品構成

元記事 ↗

Uncle Bobが提唱したSOLID原則（単一責任・依存性逆転など）を忠実にコードへ落とし込んだ構成パーツ：

Controller
ボタン押下等の入力を受け取り、コンピュータが理解できる Request Data に変換。

Input Boundary
UseCaseのインターフェース。Controllerが呼ぶ窓口。

UseCase Interactor
Input Boundaryを実装し、業務手順を実行する本体。

Data Access I/F
DBアクセスのインターフェース（Repository）。DIPで依存を逆転。

Output Boundary
Presenterが実装する出力インターフェース。

Presenter
Output Dataを受け取り、画面用のViewModelに整形する。

Part.4 処理の流れとテスト容易性、そして現場の課題

元記事 ↗

✨ **最大のメリット：DBなしで即座にテストができる**
Data Access Interface に「インメモリのスタブ」を差し込むだけで、DB起動を待たずに単体テストが書ける。 先行開発でDB選定が終わっていなくても、ビジネスロジックの検証を100%先行できる。

⚠️ **現場で直面する3つの大きな壁**
1. **MVCフレームワークとPresenterが合わない**（WebではControllerがResponseを返すのが普通）
2. **ControllerへのDI修正が頻発する**（ユースケースが増えるたびにControllerの引数が増える）

3. **とにかくクラスが多すぎて管理が大変**（インターフェースとモデルの激増）

Part.5 課題の現実的解決策：「Presenterは捨てよう」

元記事 ↗

💡 解決策1: 「Presenterは捨てる」（戻り値で返す）

教科書通りのPresenterをやめ、**「UseCase InteractorがOutput Dataを戻り値として直接返す」**形（ヘキサゴナル的スタイル）に割り切る。
これだけでSpringなどのMVCフレームワークと完全に調和し、余分なインターフェースがごっそり減る。

💡 解決策2: Message Bus の導入

Controllerに個別UseCaseを何個もDIするのをやめ、Mediator（Message Bus）を1つDIしてCommandを投げる形に集約する。

💡 解決策3: スタブ先行開発

「同じ動きをするなら実体は何でもいい」。DB接続前にメモリ上のMapでRepositoryを作り、最速で画面を動かす。

🏆 成瀬氏の結びのメッセージ

「アーキテクトの役目は、プロジェクトの進め方に責任を持つこと」。
理論通りの形に固執して現場を疲弊させるのではなく、**ビジネスを中心にして、納期と試行錯誤のトレードオフの中で最適な形に取捨選択すること**が真のクリーンアーキテクチャである。